

تعریف نرم افزار ارزشمند

علی سامانیان
810100161

آرین فیروزی
810100196

شهزاد ممیز
810100272

فهرست مطالب:

2	توضیحات و خلاصه مطالب
2	بخش اول: مقاله "What is Throughput in Performance Testing"
2	بخش دوم: مطالب مربوط به استاندارد <u>ISO/IEC 25010</u>
3	پاسخ به سوالات
4	پاسخ سوال اول
4	پاسخ سوال دوم
5	پاسخ سوال سوم
6	پاسخ سوال چهارم
7	توصیف نیازمندی ها
7	پاسخ سوال پنجم
8	پاسخ سوال ششم

1. بخش اول: مقاله " What is Throughput in Performance Testing "

مقاله «Throughput در تست عملکرد چیست؟» از وبسایت TestGuild به بررسی مفهوم «Throughput» در تست عملکرد نرم افزار می پردازد. Throughput به تعداد تراکنش هایی اشاره دارد که یک سیستم در واحد زمان پردازش می کند و معمولاً به صورت «تراکنش در ثانیه» (TPS) بیان می شود. برای توضیح این مفهوم، مقاله مثالی از یک پمپ بنزین با سه پمپ ارائه می دهد که در هر دقیقه می تواند سه خودرو را سوخت گیری کند. در این مثال، حداکثر Throughput پمپ بنزین سه خودرو در دقیقه است. به طور مشابه، اگر یک وب اپلیکیشن بتواند ۳۰ درخواست در ثانیه را پردازش کند، اما ۵۰ درخواست در ثانیه دریافت کند، ۲۰ درخواست باقی مانده در صف انتظار قرار می گیرند. مقاله همچنین به نحوه اندازه گیری Throughput در تست های عملکرد با استفاده از ابزارهایی مانند LoadRunner و JMeter اشاره می کند. در یک سناریوی تست معمولی، با افزایش تعداد کاربران، Throughput نیز افزایش می یابد. پس از رسیدن به حالت پایدار، Throughput ثابت می ماند. اگر تعداد کاربران بیش از حد افزایش یابد، ممکن است Throughput کاهش یابد که نشان دهنده وجود گلوگاه در سیستم است. در نهایت، مقاله تأکید می کند که درک صحیح از مفهوم Throughput برای انجام تست های عملکردی مؤثر و شناسایی گلوگاه های احتمالی در سیستم ضروری است.

2. بخش دوم: مطالب مربوط به استاندارد ISO/IEC 25010 :

استاندارد ISO/IEC 25010:2023 یک مدل کیفیت برای محصولات فناوری اطلاعات و ارتباطات (ICT) و نرم افزارها ارائه می دهد که شامل هشت ویژگی اصلی است:

مدل کیفیت محصول:

این مدل شامل هشت ویژگی کلیدی است که برای ارزیابی کیفیت محصول نرم افزاری استفاده می شوند:

1. مناسبیت عملکردی (Functional Suitability):

توانایی محصول در ارائه عملکردهایی که نیازهای مشخص و ضمنی کاربران را برآورده می کنند. Note 1 to entry: کامل بودن، صحت، تناسب عملکرد.

2. کارایی عملکرد (Performance Efficiency):

میزان استفاده بهینه از منابع در شرایط کاری خاص.
Note 1 to entry: زمان پاسخ، مصرف منابع، ظرفیت.

3. سازگاری (Compatibility):

توانایی محصول در تبادل اطلاعات و هماهنگی با سایر سیستم‌ها یا نرم‌افزارها.
Note 1 to entry: قابلیت تعامل، همزیستی.

4. قابلیت استفاده (Usability):

سهولت یادگیری، درک، و استفاده از محصول توسط کاربران.
Note 1 to entry: قابلیت یادگیری، کاربرپسندی، قابلیت دسترسی.

5. قابلیت اطمینان (Reliability):

توانایی محصول در ارائه خدمات پایدار و بدون خطا.
Note 1 to entry: بلوغ، تحمل خطا، بازیابی‌پذیری.

6. امنیت (Security):

توانایی حفاظت از اطلاعات و مقاومت در برابر تهدیدات امنیتی.
Note 1 to entry: محرمانگی، یکپارچگی، دسترس‌پذیری، تصدیق هویت.

7. قابلیت نگهداری (Maintainability):

سهولت در تغییر، به‌روزرسانی و نگهداری محصول.
Note 1 to entry: قابلیت تحلیل، تغییرپذیری، پایداری، قابلیت تست.

8. قابلیت حمل (Portability):

توانایی محصول در انتقال به محیط‌های مختلف.
Note 1 to entry: قابلیت سازگاری، نصب‌پذیری، جایگزینی.

این ویژگی‌ها به سازمان‌ها کمک می‌کنند تا کیفیت محصولات نرم‌افزاری خود را ارزیابی و بهبود بخشند.

پاسخ به سوالات:

با توجه به عبارت «اگر ۱۰۰۰ دانشجو در جریان فرایند انتخاب واحد باشند، سامانه باید به درخواست‌های اخذ یک واحد از طرف یک دانشجو در زمانی کمتر از ۳ ثانیه پاسخ دهد.» در صورت سوالات، به سوالات زیر پاسخ می‌دهیم

1) در مورد ترکیب داده‌ها و درخواست‌ها، چه اطلاعاتی در سناریوی فوق لحاظ نشده است که

می‌تواند روی زمان پاسخ سامانه تأثیر بگذارد؟

1. **حجم داده‌ها:** مشخص نشده است که پایگاه داده شامل چه حجمی از اطلاعات (تعداد دانشجویان، دروس، یا سوابق ثبت‌شده) است. حجم زیاد داده‌ها می‌تواند روی سرعت بازیابی اطلاعات تأثیر بگذارد.
2. **مواردی که به دیتابیس و نوع ذخیره کردن دیتا در دیتابیس است بسیار اثر گذار است** مثلاً اشاره‌ای به ساختار شاخص‌ها در پایگاه داده نشده است. استفاده یا عدم استفاده از شاخص‌ها (ایندکس‌ها) می‌تواند مستقیماً بر سرعت بازیابی داده‌ها تأثیر بگذارد. داده‌ها ممکن است در جداول مختلف یا حتی پایگاه‌های داده متفاوت ذخیره شده باشند، که بر زمان دسترسی اثر دارد.
3. **ممکن است یک درخواست نیاز به توان محاسباتی بیشتری باشد و یک درخواست سریع‌تر انجام شود.** چگونگی درخواست‌ها مشخص نیست.
4. **نوع درخواست‌ها:** مشخص نشده است که درخواست‌ها صرفاً شامل انتخاب یک واحد خاص هستند یا نیاز به بررسی پیش‌نیازها، محدودیت‌های ظرفیت، یا تطابق با برنامه دانشجوی دارند.
5. **پیچیدگی درخواست‌ها:** برخی درخواست‌ها ممکن است شامل محاسبات پیچیده یا بررسی قوانین خاص باشند که نیاز به پردازش بیشتری دارند.
6. **تعداد کاربران فعال:** ۱۰۰۰ دانشجو هم‌زمان درخواست ارسال می‌کنند، اما میزان هم‌پوشانی این درخواست‌ها یا زمان‌بندی دقیق آن‌ها مشخص نشده است. اگر تعداد زیادی درخواست به‌طور دقیق در یک لحظه خاص ارسال شوند، می‌تواند باعث ترافیک در سرور شود. و ممکن است تعدادی دانشجویان در سامانه باشند ولی هم‌زمان درخواست هایشان را ارسال نکنند. این امر بسیار مهم است که ذکر نشده.
7. **شبکه:** سرعت و تأخیر شبکه نیز می‌تواند در زمان پاسخ سامانه تأثیرگذار باشد، مخصوصاً اگر درخواست‌ها از نقاط مختلف ارسال شوند.

2) چگونه ممکن است با یک مجموعه داده آزمون (شامل داده‌های ذخیره‌شده در پایگاه داده و

محتوای درخواست‌های کاربران) زمان‌های پاسخ متفاوتی به دست آورد؟

1. **دیتابیس:** اگر مجموعه داده آزمون کوچک باشد (مثلاً فقط ۵۰ درس و ۱۰۰ دانشجو)، زمان پاسخ احتمالاً سریع‌تر از زمانی است که مجموعه داده بزرگ و واقعی باشد. به عبارت دیگر، برای دریافت اطلاعات، Query هایی که سامانه می‌زند و اینکه از چه جداول با چه دیتایی باید اطلاعات را استخراج کند، بسیار مهم است.
2. **نوع و پیچیدگی هر درخواست متفاوت است:** اگر در داده آزمون، تمامی درخواست‌ها ساده و یکسان باشند (مثلاً انتخاب یک واحد بدون پیش‌نیاز یا محدودیت)، سامانه سریع‌تر پاسخ می‌دهد. اما در داده واقعی، ممکن است درخواست‌های پیچیده‌تر وجود داشته باشد.
3. **توجه کنید که داده‌هایی که در مراحل دولوپمنت به عنوان دیتای فیک و برای صحت‌سنجی استفاده می‌کنیم ممکن است با دیتای واقعی تفاوت داشته باشد** همچنین دیتای واقعی در طول زمان تغییر می‌کند. اگر در محیط آزمون، تنها ۱۰۰ درخواست هم‌زمان شبیه‌سازی شود، زمان پاسخ کوتاه‌تر از حالتی است که ۱۰۰۰ کاربر واقعی به‌طور هم‌زمان درخواست ارسال کنند.

4. اگر داده آزمون روی پایگاه داده بهینه شده (با شاخص ها و ساختار مناسب) ذخیره شود، ممکن است نتایج بهتری ارائه دهد. اما در حالت واقعی، این بهینه سازی ممکن است ناقص باشد. و این اطلاعات را نداریم که در کدام جدولها اطلاعات به صورت بهتر و با شاخص ها ذخیره شده اند و یا تعداد ریکورد ها کمتر است و دسترسی راحتتری دارند.
5. اگر در آزمون، درخواست ها به صورت یکنواخت ارسال شوند (مثلاً هر ۱ ثانیه یک درخواست)، سامانه بهتر عمل می کند. اما در بار واقعی، درخواست ها ممکن است در زمان های خاصی متمرکز شوند (Peak Times)، که منجر به کاهش توان عملیاتی می شود.

(3) چگونه مشکل فوق را حل می کنیم؟

1. شبیه سازی لود واقعی:

- ابزارهایی مانند **JMeter** یا **LoadRunner** برای تست بارگذاری و بررسی رفتار سامانه تحت شرایط واقعی استفاده می شوند.
- این ابزارها به توسعه دهندگان کمک می کنند تا سناریوهای مختلف را بررسی کنند، مانند افزایش تدریجی تعداد کاربران تا رسیدن به آستانه بحرانی، شناسایی زمان پاسخ سامانه، و بررسی ظرفیت تحمل سیستم.
- بدین ترتیب می توان رفتار سامانه را در **Peak Time** شبیه سازی کرد و تأثیر گلوگاه ها را بررسی و رفع کرد.

2. بهینه سازی پایگاه داده:

- **ایجاد شاخص ها (Indexes):** شاخص ها جستجو در پایگاه داده را سرعت می بخشند. مثلاً اگر دانشجو بر اساس کد ملی جستجو شود، ایجاد یک شاخص روی ستون "کد ملی" بازیابی اطلاعات را سریع تر میکند.
- **استفاده از کش (Caching):** داده های پرتکرار (مانند لیست دروس و ظرفیت آن ها) را در حافظه موقت نگه می داریم تا سامانه مجبور به دسترسی مکرر به پایگاه داده نشود.
- **نرمال سازی و دنرمال سازی:** برای کاهش تکرار داده ها و افزایش کارایی، جداول پایگاه داده را بهینه می کنیم. مثلاً برای ثبت اطلاعات دانشجویان، جداول جداگانه برای اطلاعات شخصی و آکادمیک ایجاد می کنیم.

3. بهینه سازی معماری نرم افزار:

- **Load Balancer:** این تکنیک درخواست های کاربران را به سرورهای مختلف تقسیم می کند. در نتیجه، یک سرور تحت فشار زیاد قرار نمی گیرد.
- **مثال:** وقتی ۱۰۰ درخواست دریافت می شود، ۵۰ درخواست به سرور A و ۵۰ درخواست به سرور B منتقل می شود.
- **مقیاس پذیری افقی:** افزودن سرورهای جدید به شبکه.
- **مقیاس پذیری عمودی:** ارتقاء سخت افزار یک سرور (مثل افزایش RAM یا پردازنده).

4. بهینه سازی کد و درخواست ها:

- **ساده سازی درخواست ها:** درخواست هایی که نیاز به بررسی پیش نیاز یا ظرفیت دروس ندارند، مستقیم پاسخ داده شوند.
- **مثال:** اگر دانشجو تنها قصد مشاهده دروس را دارد، نیازی به بررسی ظرفیت کلاس نیست.
- **پردازش دسته ای (Batch Processing):** درخواست های مشابه را گروه بندی کرده و با یک Query پاسخ دهید.
- **مثال:** اگر چند دانشجو قصد اخذ یک درس را دارند، به جای بررسی تک تک، ظرفیت را یک بار کلی بررسی کنید.

(4) به نظرتان حل این مشکل وظیفه مشتری است یا توسعه دهنده.

عملاً مقصر این مشکلات را نمیتوان به طور صریح مشخص کرد و نظر قطعی داد، حال در ادامه به بررسی وظایف مشتری و توسعه دهنده میپردازیم و راهکار هایی که میتواند از بروز چنین مشکلاتی جلوگیری کند ارائه میدهیم.

وظایف توسعه دهنده:

در سوال 3 البته کمی مرور شده این موضوع و اینجا گذرا آنرا بررسی میکنیم.

1. تحلیل و طراحی بهینه سامانه:

توسعه دهنده باید بر اساس نیازهای مشتری، سامانه ای طراحی کند که بتواند معیارهای مشخص شده مانند "پاسخ کمتر از ۳ ثانیه برای ۱۰۰۰ کاربر همزمان" را برآورده کند.

مثال: اگر مشتری تعداد کاربران همزمان را اعلام کرده است، توسعه دهنده باید این عدد را مبنای طراحی و تست سامانه قرار دهد.

2. اجرای تست های عملکرد:

توسعه دهنده موظف است سامانه را تحت سناریوهای واقعی (شبیه سازی تحت شرایط واقعی با ابزارهایی مانند JMeter) بررسی کند. در این مرحله، گلوگاه ها و مشکلات احتمالی شناسایی شده و رفع می شوند.

3. ارائه راهکارهای بهینه سازی:

اگر محدودیت ها مربوط به زیرساخت یا منابع سخت افزاری باشد، توسعه دهنده باید این مسائل را به مشتری اعلام کند و راهکارهایی برای بهبود ارائه دهد.

مثال: پیشنهاد استفاده از سرورهای قوی تر یا افزودن سرورهای جدید.

4. تضمین مقیاس پذیری سامانه:

سامانه باید به گونه ای طراحی شود که در آینده قابلیت افزایش کاربران یا داده ها را بدون افت عملکرد داشته باشد.

وظایف مشتری:

1. تعریف شفاف نیازمندی ها:

مشتری باید تمام جزئیات موردنظر خود را در اختیار توسعه دهنده قرار دهد، مانند تعداد کاربران همزمان، حجم داده ها، نیازمندی های عملکردی (مثل زمان پاسخ)، و سناریوهای عملیاتی.

مثال: مشخص کند که ۱۰۰۰ کاربر دقیقاً چه نوع درخواست هایی ارسال می کنند (مثل اخذ واحد، بررسی پیش نیاز).

2. تأمین منابع و زیرساخت مناسب:

اگر سامانه نیاز به سرورهای قوی تر یا پهنای باند بیشتر (پایگاه داده سریع تر) باشد، مشتری باید این منابع را فراهم کند.

مثال: اگر سرور فعلی توانایی پشتیبانی از تعداد کاربران را ندارد، مشتری وظیفه دارد سرور جدید خریداری کند.

3. مشارکت در فرآیند تست:

به طور کلی بهتر است مشتری در فرآیند های تست نرم افزار مشارکت داشته باشد و همچنین موارد کلیدی را در زمان بررسی اسپرینت ها مورد بحث قرار دهند.

مشتری میتواند داده های واقعی تری را برای شبیه سازی سامانه در اختیار توسعه دهنده قرار دهد. همچنین، بازخورد مشتری پس از استفاده آزمایشی میتواند به بهبود سامانه کمک کند.

4. پذیرش مسئولیت در شرایط خاص:

اگر مشکل ناشی از عوامل خارجی مانند تأخیر شبکه یا کمبود منابع باشد، چنانچه همان مشتری ارائه کننده سرویس مد نظر باشد باید این عوامل را مدیریت کند.
مثال: ارتقاء پهنای باند اینترنت برای کاربران از نقاط جغرافیایی دورافتاده (مشتری: ارائه دهنده سرویس های مخابراتی).

توصیف نیازمندی های جنبه های مختلف:

فرض کنید شما سفارش دهنده نسل بعدی سامانه انتخاب واحد هستید و می خواهید برای تیم توسعه دهنده نیاز خود را توصیف کنید. به ازای هر یک از جنبه های زیر که از استاندارد ایزو استخراج شده اند

(5) حداقل یک نیازمندی کیفیت برای کارکرد انتخاب واحد از سامانه جامع آموزش توصیف کنید.
دقت کنید برای هر جنبه می توان چندین نیازمندی داشت که شما حداقل یک نمونه را توصیف کنید.

1. **نیازمندی performance:** با توجه به اینکه تعداد دانشجویانی که از این ابزار استفاده میکنند زیاد است و میزان درخواستهایی که سامانه متحمل میشود تقریباً به صورت توزیع پواسون است، باید سامانه را طوری طراحی کنیم که بتواند بار زیاد درخواستها را جوابگو باشد وگرنه غیر قابل استفاده خواهد بود.
2. **نیازمندی compatibility:** این سامانه باید با سامانه احراز هویت یکپارچه باشد تا دانشجویان بتوانند به اکانت خود وارد شوند.
3. **نیازمندی interaction:** سامانه جدید نباید از لحاظ رابط گرافیکی فرق قابل توجهی با سامانه قبلی داشته باشد، هر چند میتوانیم تغییرات جزئی و ویژگی های جدید اضافه کنیم ولی باید توجه کنیم که عملکرد های قبلی سامانه به نحوی در سامانه جدید ایجاد شوند که کاربر مسیر یکسانی را برای آن طی کند.
4. **نیازمندی reliability:** باید انتخابهای انجام شده توسط دانشجویان در حین انتخاب واحد realtime ثبت شوند و در صورت بروز هر نوع خطایی به کاربر خطا نشان دهد. همچنین باید حالت احراز هویت با چند دستگاه مختلف را در نظر بگیریم و فرد نتواند با چند دستگاه انتخاب واحد انجام دهد چون state سیستم در دو دستگاه متفاوت میشود.
5. **نیازمندی security:** سیستم باید در مقابل حملات (مثل ddos) مقاوم باشد. همچنین یکپارچه سازی با سامانه احراز هویت باید به صورت امن انجام گیرد.
6. **نیازمندی flexibility:** چون تعداد زیادی از دانشجویان با دستگاه های مختلف وارد میشوند، باید در محیط های مختلف قابلیت اجرا داشته باشد.

(6) دقت نمایید تمامی سناریوهای کیفیت مطرح شده باید به طور کمی قابل درستی سنجی باشند.

برای هر یک از نیازمندی‌های توصیف شده نحوه درستی سنجی آن را هم تشریح کنید.

1. **درستی سنجی performance:** سیستم را تحت بار زیاد (به تعداد نزدیک به دانشجویان) اجرا میکنیم و یک لیمیت زمانی برای دریافت پاسخ برای ریکوئست ها ست میکنیم.
2. **درستی سنجی compatibility:** نوع درستی سنجی این مورد صرفا یک مقدار boolean برای امکان لاگین کردن است. میتوان این امکان را برای یک دانشجو و یک مدیر سایت تست کرد.
3. **درستی سنجی interaction:** این مورد به دانشجو بستگی دارد و user acceptance است، در نتیجه باید از تعدادی دانشجویان استفاده کنیم. برای مثال میتوان نسخه بتا را قبل از ریلیز اصلی ران کرد و به تعدادی دانشجو access داد تا سامانه را تست کنند. برای کمی کردن این مورد میتوان فرمی برای رضایت از بخش های مختلف با فیدبک عددی ارائه داد.
4. **درستی سنجی reliability:** با تست های حالت خاص میتوان این مورد را انجام داد ولی به علت ذات concurrent بودن بعضی خطا ها ممکن است این بخش مقداری دشوار باشد. تلاش میکنیم خطا های مختلف را بازسازی کنیم و تست کنیم که سیستم برای تمامی این حالت ها ارور مناسب نشان دهد و به حالت قبل از خطا برگردد.
5. **درستی سنجی security:** این مورد نیز به آسانی قابل صحت سنجی نیست و هیچوقت نمیتوانیم صد درصد از امنیت سایت مطمئن شویم، ولی میتوان معیاری برای آن تعریف کرد. برای مثال میتوان با یک متخصص نفوذ در این مورد مشورت کرد و یک سری vulnerability را کاور کرد.
6. **درستی سنجی flexibility:** ابتدا لیست محیط های رایج را در نظر میگیریم و در صورتی که وقت داشته باشیم لیست را کاملتر میکنیم، سیستم را در محیط های مختلف تست میکنیم.